EE2026: DIGITAL DESIGN

Academic Year 2026-2027, Semester 1

LAB 4: Sequential Circuits in Verilog

OVERVIEW

A sequential circuit is one where the outputs depend on the current inputs and the sequence of past inputs. As a result, a sequential circuit has memory, also called states. In this lab, some basic sequential circuits will be designed to make an LED blink at various speeds.

The pre-requisites for this lab are:

- A very good understanding and application of dataflow modelling and structural modelling in designing modules.
- Familiarity and knowledge on how to use "*Set as Top*", "*reg*" and "*wire*".

This lab will cover the following:

- Using a signal that inverts itself periodically, which shall be called **CLOCK**.
- Making a physical LED blink by using the Basys 3 development board clock signal.

Tasks for this lab include:

- Creating a slower clock from a faster clock.
- Having a physical LED blink noticeably on the Basys 3 development board, by using the slower clock.
- Using switches to make a physical LED blink at different speeds on the Basys 3 development board.

GRADED PRACTICE TASK [CANVAS SUBMISSION REQUIRED]:

Details for GPT-4 are available at the end of this lab manual.

PREPARATION FOR THE FPGA DESIGN PROJECT (WEEK 7 to WEEK 12):

Students are required to form their own team of 4 and register during the lab session in week 7. Depending on class size, we may need to create one to three teams of 3. We will arrange for all students without a team to meet each other during the lab session in week 7. Then you can form your own team after checking with each other.

Teams that consist of students who have not submitted GPT-4 will not be automatically approved, and such students may be required to work individually. Teams should confirm with all their members if they have submitted GPT-4 before registering their team in week 7.

THE BLINKING LED

A simple blinking LED is required to be implemented on the FPGA. A new signal, **CLOCK**, will be introduced.

The **CLOCK** signal is an external input signal that resembles a square wave of 50% duty cycle. If this **CLOCK** signal is connected directly to a physical LED, the latter will light up when the signal is HIGH, and will switch off when the signal is LOW, as illustrated in **Figure 3.1**.

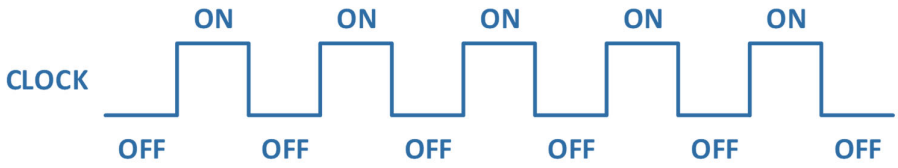


Figure 3.1: *A **CLOCK** signal with 50% duty cycle*

A simple dataflow description in Verilog for a blinky module is written first, followed by a simulation source to verify the design. To create the square wave, or **CLOCK** signal, in the simulation source, a new section of codes will now be introduced:

Verilog code for blinky, using the dataflow method

```
module blinky (input CLOCK, output LED);

    assign LED = CLOCK;

endmodule
```

Simulation source code to test the blinky design

```
`timescale 1ns / 1ps

module test_blinky( );

    reg CLOCK; wire LED;

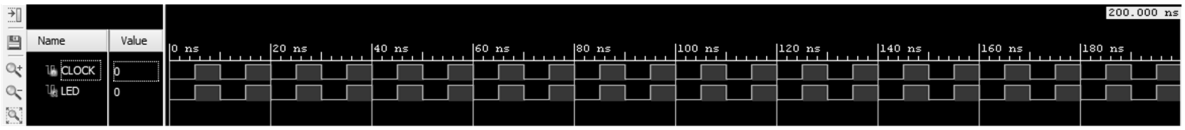
    blinky dut (CLOCK, LED);

    initial begin
        CLOCK = 0;
    end

   always begin
       #5; CLOCK = ~CLOCK;
   end

endmodule
```

Expected simulation waveform for the blinky design



UNDERSTANDING | TASK 1

Based on the Verilog code and simulation results, check your understanding by answering the following questions:

1. Every 5 units of time, **CLOCK** is being inverted. What is the frequency of that signal being created in simulation?

   _____

2. What would happen if the testbench code **CLOCK = 0** is removed?

   _____

For the hardware implementation, instead of using an external signal generator for the **CLOCK** signal to the Artix-7 FPGA, the Basys 3 development board includes a single 100 MHz clock generator connected to pin W5 of the Artix-7 FPGA.

Using the original contents of the ***Basys3_Master.xdc*** in your constraint file, follow these steps:

1. Uncomment lines 7 and 8 to use the 100 MHz on the Basys3 development board. If required, rename the signal to the name used in your **blinky** code. In our example, the name **CLOCK** was used, and the final changes may look similar to **Figure 3.2**.

2. Configure the output signal **LED** that is present in your **blinky** code (or the name chosen by you while writing the code) by linking it to any physical LED on the Basys3 development board.

```
1  ## This file is a general .xdc for the Basys3 rev B board
2  ## To use it in a project:
3  ## - uncomment the lines corresponding to used pins
4  ## - rename the used ports (in each line, after get_ports) according to the top level signal names in the project
5
6  # Clock signal
7  set_property -dict { PACKAGE_PIN W5   IOSTANDARD LVCMOS33 } [get_ports CLOCK]
8  create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports CLOCK]
9
10
11 ## Switches
12 #set_property -dict { PACKAGE_PIN V17   IOSTANDARD LVCMOS33 } [get_ports {sw[0]}]
13 #set_property -dict { PACKAGE_PIN V16   IOSTANDARD LVCMOS33 } [get_ports {sw[1]}]
```

Figure 3.2: *Modifying your constraint file, based on the contents of the Basys3_Master.xdc*

UNDERSTANDING | TASK 2

You may optionally generate the bitstream and download your code to the Basys3 development board. What do you *"notice"* about the "*blinking*" LED?

_____

_____

COUNTER FOR A SLOWER CLOCK

To be able to observe a blinking LED at a frequency that is visible to the human eyes, modifications need to be done to the Verilog code to have a slower clock. Let us introduce a temporary variable **COUNT** that is incremented by 1 at every rising edge (transition from low to high; also called a positive edge) of the **CLOCK** signal, as shown in **Figure 3.3**.

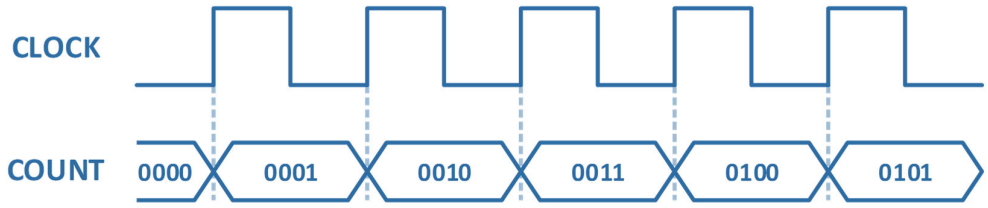


Figure 3.3: *Increasing* ***COUNT*** *at each rising edge of* ***CLOCK***

Verilog code for an incrementing counter of 4-bit, using behavioural modelling

```
module slow_blinky_module (input CLOCK);

    reg [3:0] COUNT = 4'b0000;

    always @ (posedge CLOCK) begin
        COUNT <= COUNT + 1;
    end

endmodule
```

Now, use a conditional statement that changes the state of an output signal, called SLOW_CLOCK, whenever COUNT reaches zero. Here, since the 4-bits COUNT has 16 possible states, it would mean that SLOW_CLOCK changes state after 16 clock cycles of CLOCK. In other words, SLOW_CLOCK is 1/32th the clock speed of CLOCK.

Partial Verilog code (Determine what signal to declare!) for toggling SLOW_CLOCK signal, using behavioural modelling

```
    always @ (posedge CLOCK) begin
        COUNT <= COUNT + 1;
        SLOW_CLOCK <= ( COUNT == 4'b0000 ) ? ~SLOW_CLOCK : SLOW_CLOCK ;
    end
```

UNDERSTANDING | TASK 3

Complete the partial code above for the SLOW_CLOCK signal so that it can be used for implementation / simulation.

Hint: SLOW_CLOCK is an output signal, as well as a register that holds the SLOW_CLOCK value. The initial state of the SLOW_CLOCK register can be zero or one.

Simulate the Verilog codes, so that a waveform that toggles every 16th clock cycle can be observed.

UNDERSTANDING | TASK 4

Explore what happens if COUNT is forcefully toggled before the overflow to 0 occurs. To do so, add a conditional statement to allow COUNT to become 0 at an earlier clock cycle instead of waiting for 16 clock cycles.

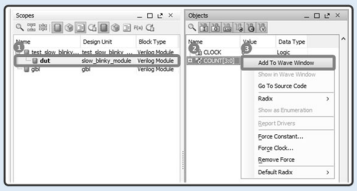
```
    always @ (posedge CLOCK) begin
        COUNT <= (COUNT == 4'b1000) ? 0 : COUNT + 1;
        SLOW_CLOCK <= ( COUNT == 4'b0000 ) ? ~SLOW_CLOCK : SLOW_CLOCK ;
    end
```

Simulate the Verilog codes again and observe if the waveforms have toggled earlier.

FURTHER ANALYSIS IN THE SIMULATION WAVEFORM WINDOW

By default, the simulation window only shows the waveforms of input and output signals. To see the waveforms of variables during the simulation, such as the variable **COUNT**:

1. Select the test unit for the simulation module being used
2. In the **Objects** window, right click on the **COUNT** variable
3. Choose **Add To Wave Window**



After adding the variable **COUNT** to the wave window, the current simulation needs to be re-run. Follow these steps:

1. Click on the "Rewind" symbol to remove the waveforms
2. Set the simulation time and unit
3. Click on the "Run" symbol to simulate for the time set in step 2

This is an important step to remember if you want to simulate for more than the default 1000 ns of simulation!!!



The **COUNT** variable can then be expanded by clicking on the **+** symbol to the left of **COUNT**. This allows for every individual bit to be observed as independent waveforms:



UNDERSTANDING | TASK 5

Assume LD15 on the Basys3 development board is the LED to blink.

Consider the three switches SW0, SW1 and SW2, and their behaviours:

- When these three switches are OFF, LD15 must be OFF
- When SW0 is ON, LD15 must blink at 30 Hz
- When SW1 is ON, LD15 must blink at 3 Hz
- When SW2 is ON, LD15 must blink at 1 Hz

Implement the above requirement on the Basys 3 development board. Use the following hints:

- Create a top-level module with the Basys3 clock and 3 switches as inputs, and 1 LED as output
- Create **3 separate Verilog modules**, each one producing a slower clock speed of 30 Hz, 3 Hz and 1 Hz
- The 3 slower clock signals go into a multiplexer. The output of the mux connects to the LED
- The multiplexer has a 3-bits control signal. Based on the switch that is on, one of the slower clock speeds will be connected to the output of the mux. A value of 0 connects to the output if all three switches are off

Note: For the EE2026 labs, a blinking frequency of 1 Hz means that the LED should be ON for 0.5 seconds, and OFF for around 0.5 seconds, for each blinking cycle

UNDERSTANDING | TASK 6

Assume an LED on the Basys3 board needs to turn on every one second:

When 1 second has passed since the beginning, only LD0 turns ON.
When 2 seconds have passed since the beginning, only LD1 turns ON.
When 3 seconds have passed since the beginning, only LD2 turns ON.
When 4 seconds have passed since the beginning, only LD3 turns ON.

To implement the above task on the Basys3 development board, use the following:

- Create a top-level module with the basys3 clock as input, and 4 LEDs as outputs.

- Create a module whose only purpose is to increase a COUNT value every 1 second. A **partial Verilog code** is:

```
always @ (posedge CLOCK_1HZ) begin
    COUNT <= COUNT + 1;
end
```

  In other words, COUNT is keeping track of time, accurate to 1 second.

- Create another module whose only purpose is to control the LEDs. A **partial Verilog code** is:

```
always @ (posedge CLOCK_25MHZ) begin
    if (COUNT == 2'b00) begin
       LEDS <= 4'b0001;
    end
    if (COUNT == 2'b01) begin
       LEDS <= 4'b0010;
    end
    if (COUNT == 2'b10) begin
       LEDS <= 4'b0100;
    end
    if (COUNT == 2'b11) begin
       LEDS <= 4'b1000;
    end
end
```

- Use the top-level module to instantiate and connect the **separate modules** that you have created above. You can now implement that top-level module on the Basys3 board to see a certain LED turning ON every 1 second.

IMPORTANT: It is required to **successfully implement and fully understand the structural modelling and behavioural codes** used in **UNDERSTANDING | TASK 5** and **UNDERSTANDING | TASK 6** **before starting** with GPT-4.

OPTIONAL KNOWLEDGE

Suppose we want to create an LED that blinks at 6.25 MHz.

Two possible options we can consider are indicated below:

<table>
<tr><th>CLOCK DIVIDER METHOD</th><th>CLOCK ENABLE METHOD</th></tr>
<tr><td>

```
// Partial codes
// Declare input, output, registers

always @ (posedge basys_clock)
begin
    count <= (count == 7) ? 0 : count + 1;
    new_clk <= (count == 0) ? ~new_clk : new_clk;
end

assign my_led = new_clk;
```

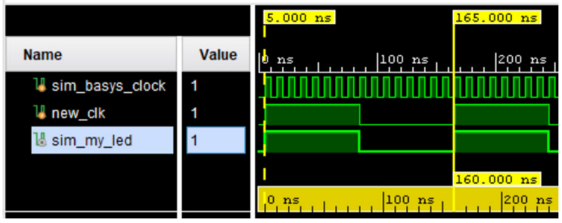
</td><td>

```
// Partial codes
// Declare input, output, registers

always @ (posedge basys_clock)
begin
    count <= (count == 7) ? 0 : count + 1;
    enable <= (count == 7) ? 1 : 0;

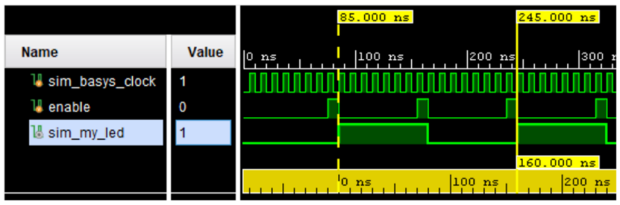
    if (enable) my_led <= ~my_led;
end
```

</td></tr>
<tr><td>



Time period = 160 ns
Frequency = 6.25 MHz

</td><td>



Time period = 160 ns
Frequency = 6.25 MHz

</td></tr>
<tr><td>

Timing report can be obtained in the "**Design Runs**" Tab

Status	WNS	TNS	WHS	THS
synth_design Complete!				
write_bitstream Complete!	5.811	0.000	0.201	0.000

</td><td>

Timing report can be obtained in the "**Design Runs**" Tab

Status	WNS	TNS	WHS	THS
synth_design Complete!				
write_bitstream Complete!	5.243	0.000	0.190	0.000

</td></tr>
<tr><td colspan="2">

Clocking considerations in an FPGA is a complex process that requires an understanding of the clock management tile (CMT). In 7 series FPGAs, the clock management tile (CMT) includes a mixed-mode clock manager (MMCM) and a phase-locked loop (PLL). For our Basys3, the documentation can be found at:

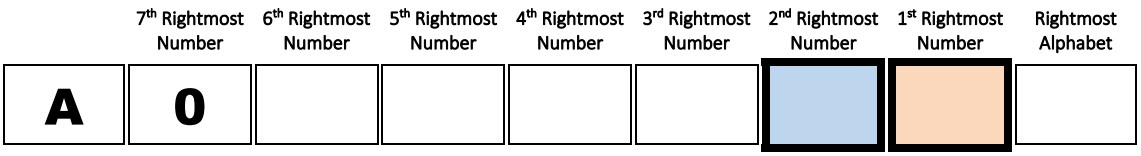
https://docs.amd.com/v/u/en-US/ug472_7Series_Clocking

In EE2026, we do not cover the clocking aspects of FPGA programming!

</td></tr>
</table>

BEFORE STARTING ON GPT-4:

Each student has a personalised requirement based on student matriculation number. Carefully write down and use your exact number, as mistakes in the student matriculation number are not accepted. Appeals due to wrong number used for your personalised requirement will be rejected.



Students are required to carefully read all the GPT-4 requirements, and to enhance their understanding by looking at the example.

Marks are given for using your correct student matriculation number, and grading is done only once. Re-submissions, late submissions, or updates, even if very simple or minor, are **not accepted** after the grading.

GRADED PRACTICE TASK 4 (GPT-4)

Complete as much as possible, in **ONE working bitstream for this GPT-4**. It is much better to have a working program with some completed functionalities, instead of submitting a program without a working bitstream (No marks given).

IMPORTANT CHARACTERS

In GPT-4, these are the important characters to note from your student matriculation number:

- The 1st rightmost numerical value of your student matriculation number (Subtask A)
- The 2nd rightmost numerical value of your student matriculation number (Subtask B)

INITIALISATION

When the program starts, switches SW0 to SW15 must be in the OFF position. The seven segment displays are all OFF.

SUBTASK A

At the start of the program, a set of LEDs starting from LD0 to MAX_LED (Including) are required to turn ON after every TIME_COUNT. Other LEDs, from MAX_LED (Excluding) to LD15, are OFF. MAX_LED is dependent on the **1st rightmost numerical value of your student matriculation number**, as indicated in the table below:

1st Rightmost Numerical Value	MAX_LED	TIME_COUNT
0	LD14	0.35 seconds (Error of ± 0.02 seconds accepted)
1	LD13	0.50 seconds (Error of ± 0.02 seconds accepted)
2	LD12	0.64 seconds (Error of ± 0.02 seconds accepted)
3	LD11	0.85 seconds (Error of ± 0.02 seconds accepted)
4	LD10	1.62 seconds (Error of ± 0.04 seconds accepted)
5	LD9	2.05 seconds (Error of ± 0.04 seconds accepted)
6	LD8	1.78 seconds (Error of ± 0.04 seconds accepted)
7	LD7	1.75 seconds (Error of ± 0.04 seconds accepted)
8	LD6	1.82 seconds (Error of ± 0.04 seconds accepted)
9	LD5	0.67 seconds (Error of ± 0.02 seconds accepted)

Use a stopwatch to verify that the total amount of time it takes for the LEDs to light up from LD0 to MAX_LED is what you expected!

After all the LEDs from LD0 to MAX_LED are ON, the user can turn ON certain switches. These switches should make certain LEDs blink, as indicated in the truth table below (**X is the don't care condition**):

INPUTS				**OUTPUTS**			
SW15 to SW5, SW1, SW0	**SW4**	**SW3**	**SW2**	**MAX_LED to LD5, LD1, LD0**	**LD4**	**LD3**	**LD2**
X	0	0	0	ON	ON	ON	ON
X	0	0	1	ON	ON	ON	Blink at 0.25 Hz
X	0	1	X	ON	ON	Blink at 5 Hz	ON
X	1	X	X	ON	Blink at 23 Hz	ON	ON

(Error of ± 5% accepted for all blinking frequencies)

Note: *The grader will only ON the switches after MAX_LED is ON. The switches will not be ON while the LEDs are lighting up.*

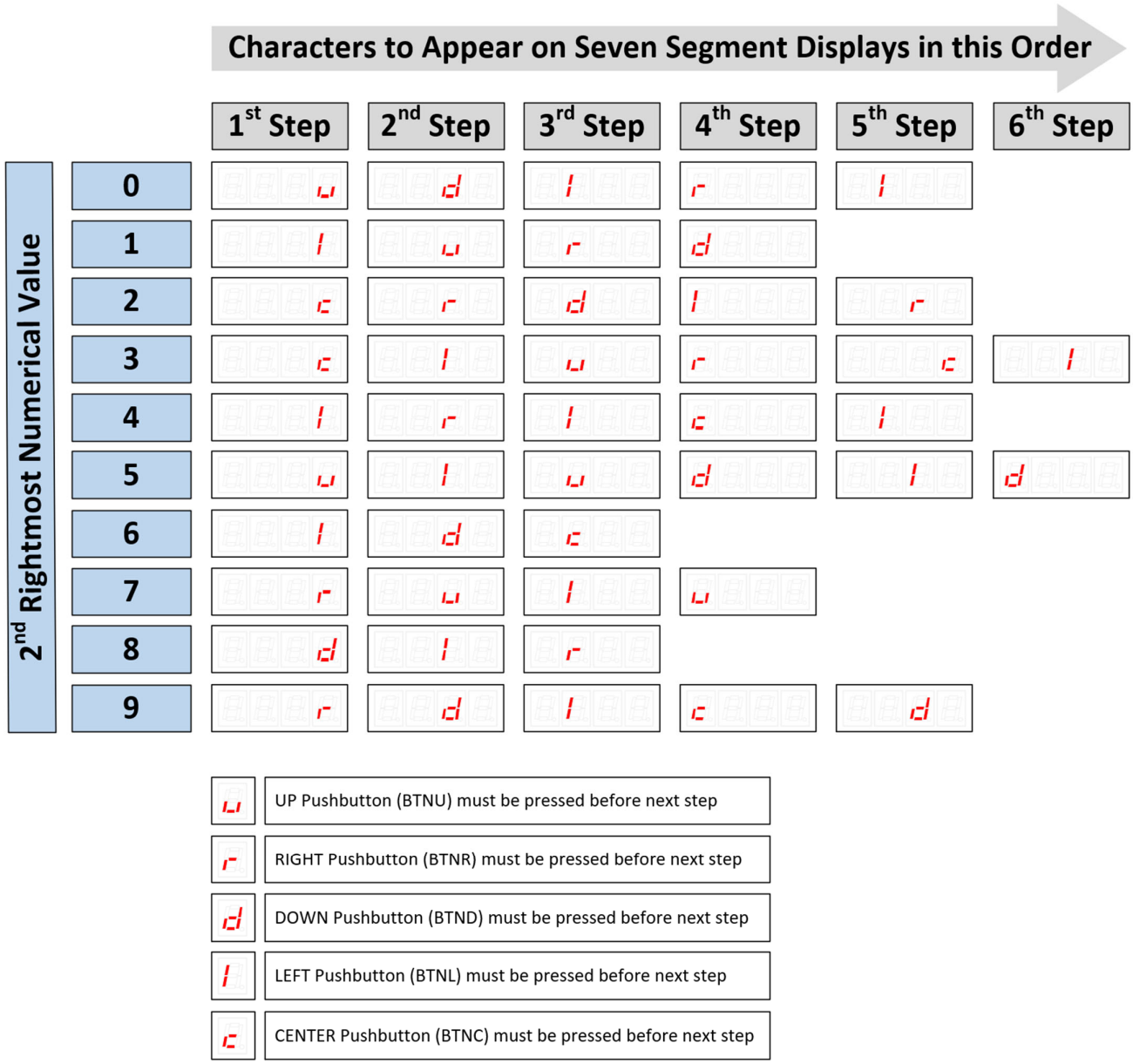
SUBTASK B

The seven segment displays must not show anything before MAX_LED is ON.

At the end of SUBTASK A, the seven segment displays and MAX_LED simultaneously turn ON (Both seven segments and MAX_LED are to be ON within 20 milliseconds of each other).

The seven segment displays start by showing the "1st Step" character. The user will then be required to press certain pushbuttons based on the characters that appear on the seven segment displays. The anodes that the characters are supposed to be displayed on are different for each step (They are not on the same anodes at each step).

All pushbuttons' presses must always be detected within 20 milliseconds of being pressed. In other words, the system must react to actions very responsively, and by changing to the next step within 20 milliseconds.



After the user has pressed the required pushbuttons in the correct order, the system goes into an "UNLOCKED" mode and stays there forever, which is indicated by LD15 being ON. The seven segment displays must then show the 1st Step again.

Note:

- *The system should ignore all incorrect presses (Nothing happens for wrong pushbutton presses)*
- *The pushbuttons will not be pressed after the system goes into "UNLOCKED" mode.*

IMPORTANT REMINDERS:

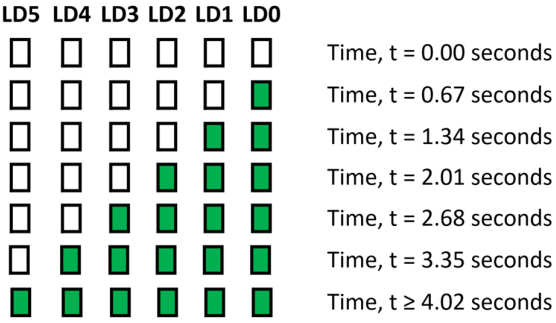
- Case statements and if-else statements must be inside an always block.
- **Be careful of parallel execution of the always blocks. Multi-driven nets indicate that there are conflicting values being given to the same signal from different always blocks.** For example, one cannot tell a signal to increase at a time instant t, and at that same time instant t, telling it to decrease.

EXAMPLE OF GPT-4 BEHAVIOUR:

If your student matriculation number is A0159089Y, then:

1st rightmost numerical value: **9** (Subtask A depends on this number: MAX_LED is LD5)
2nd rightmost numerical value: **8** (Subtask B depends on this number)

SUBTASK A



For this student (A0159089Y), MAX_LED is LD5.
Since MAX_LED is ON when t ≥ 4.02 seconds for this student, LEDs can now blink as indicated in the truth table:

INPUTS				OUTPUTS			
SW15 to SW5, SW1, SW0	SW4	SW3	SW2	MAX_LED to LD5, LD1, LD0	LD4	LD3	LD2
X	0	0	0	ON	ON	ON	ON
X	0	0	1	ON	ON	ON	Blink at 0.25 Hz
X	0	1	X	ON	ON	Blink at 5 Hz	ON
X	1	X	X	ON	Blink at 23 Hz	ON	ON

SUBTASK B

For this student (A0159089Y), MAX_LED is LD5:

All pushbuttons' presses must always be detected within 20 milliseconds of being pressed. In other words, the system must react to actions very responsively, and by changing to the next step within 20 milliseconds.



LD5 is still OFF.
Therefore, seven segments displays are still OFF.

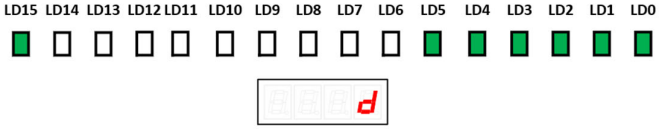
Seven segments displays update within 20 milliseconds when LD5 turns ON.

User has just pressed BTND in the previous step.

User has just pressed BTNL in the previous step.

User has just pressed BTNR in the previous step.
System now in "UNLOCKED" mode.

In "UNLOCKED MODE", LD15 is ON and the seven segment displays show the 1st step again. The pushbuttons will no longer be pressed. Switches from SUBTASK A will still be used.



CANVAS SUBMISSION INSTRUCTIONS

- Ensure that your bitstream has been successfully generated and tested on your Basys 3 development board **BEFORE** archiving your Vivado workspace for CANVAS upload.

- It is compulsory to archive your project in a compressed form without any saved simulation waveforms. Follow the instructions given in the pdf: "Archive Project in Vivado 2025.2". **The archive size should not exceed 5 MB in size for GPT-4.** If the archive size exceeds, ensure that there is no ".sim" folder in your Vivado project folder before archiving.

- **After** following the instructions in "**Archive Project in Vivado 2025.2**", rename your project archive as indicated in the appendix of this lab manual.

- Upload to CANVAS EE2026 -> Assignments -> Lab 4 -> GPT-4 Submission (On-Time).

- Download your CANVAS archive after uploading. **Click and drag the single folder within that archive to desktop**, and then open the Vivado project (.xpr) in that **extracted folder** to see if it can be opened. You can ignore "out-of-date" warnings. **Check if you can also run your bitstream correctly.** No project file and no working bitstream is equivalent to losing all marks.

- GPT-4 upload must be completed by **Saturday 26th September 2026, 6:00 A.M.** Avoid planning to upload during the additional grace period of 2 hours. (Grace period is for last-minute cases, such as due to slow internet, wrong uploads etc.)

- The late submission folder closes 1 week after the original deadline. **Late submissions are not graded if submissions are found in the on-time folder. Do not submit in the late submission folder if you have already submitted in the on-time folder, as it will be ignored and only the on-time submission will be graded.** The late submission folder will be located at: CANVAS EE2026 -> Assignments -> Lab 4 -> GPT-4 Submission (Late).

- Re-submissions, late submissions, or updates, even if very simple or minor, are **not accepted** after your original submission has been graded.

- Submissions through emails, file sharing programs, or links, are **not accepted**. Submissions must be officially submitted to, and downloadable from, Canvas in **the correct on-time or late folder**.

Plagiarism is penalised with a 100% penalty for all SOURCES and RECIPIENTS

All past and future submissions, and marks, will be reviewed in greater detail, for any person found to have plagiarised

ALL THE SUBMISSION INSTRUCTIONS LISTED ABOVE WILL AFFECT YOUR GRADES!

GRADING PROCESS

Submissions not following all the ***CANVAS SUBMISSION INSTRUCTIONS*** (listed above) will not be graded immediately, and they will instead be reviewed towards the end of the semester. **You will not be able to see your submission results during the lab sessions in such situations**.

APPENDIX (COMPULSORY renaming just before CANVAS upload):

It is **compulsory to rename your project archive** just before the CANVAS upload, as listed in the table below.

Do not change any other part of the naming. Simply **copy** the naming from the table below, and **paste** it while renaming your project archive. Penalties will be incurred if your submission cannot be found according to the exact naming template below.

CANVAS may automatically add a suffix (Example. "-1", "-2", "-3" etc.) at the end of the filename if you submit multiple files. That is acceptable and we will grade the latest file submitted within that on-time folder.

Name	Archive Naming (.xpr.zip and .zip accepted)
AARON CHIN JIN HAO	GPT4_AARON CHIN JIN HAO_163_Archive
ADI RAMANLAL	GPT4_ADI RAMANLAL_033_Archive
ADVAIT BAGRI	GPT4_ADVAIT BAGRI_209_Archive
AHMED IMAAD	GPT4_AHMED IMAAD_686_Archive
ALYCIA YAK SHI YI	GPT4_ALYCIA YAK SHI YI_246_Archive
AMBAR	GPT4_AMBAR_974_Archive
ANDREW CHONG KIAN WEI	GPT4_ANDREW CHONG KIAN WEI_019_Archive
ANDREW YAP ZI ZEN	GPT4_ANDREW YAP ZI ZEN_117_Archive
ARNAV PARAG SALKADE	GPT4_ARNAV PARAG SALKADE_052_Archive
ARORA JITISHA	GPT4_ARORA JITISHA_189_Archive
ARUN PANCHAPAKESAN IYER	GPT4_ARUN PANCHAPAKESAN IYER_842_Archive
ATHARVA REGE	GPT4_ATHARVA REGE_841_Archive
BALAKRISHNAN SHRIKKANTH	GPT4_BALAKRISHNAN SHRIKKANTH_980_Archive
BARANIKUMAR SARITHA PADMANABAN	GPT4_BARANIKUMAR SARITHA PADMA_345_Archive
BETTS DANIEL AN-ZHE	GPT4_BETTS DANIEL ANZHE_421_Archive
BHAO TANUSH	GPT4_BHAO TANUSH_706_Archive
BI JIALIN	GPT4_BI JIALIN_083_Archive
BODAPATI VAMSHI KRISHNA	GPT4_BODAPATI VAMSHI KRISHNA_405_Archive
BRYAN CHUA XIANG YU	GPT4_BRYAN CHUA XIANG YU_710_Archive
CAI YI HANG	GPT4_CAI YI HANG_455_Archive
CHAN CHEE HENG	GPT4_CHAN CHEE HENG_721_Archive
CHANG YICHUN	GPT4_CHANG YICHUN_522_Archive
CHANG ZI SHYAN, SHERWIN	GPT4_CHANG ZI SHYAN SHERWIN_293_Archive
CHEN HOK LEONG	GPT4_CHEN HOK LEONG_601_Archive
CHEN ZHENONG	GPT4_CHEN ZHENONG_159_Archive
CHEW WAI JUN EVAN	GPT4_CHEW WAI JUN EVAN_524_Archive
CHIA MING HENG, TITUS	GPT4_CHIA MING HENG TITUS_200_Archive
CHUA CHOON HSIANG	GPT4_CHUA CHOON HSIANG_727_Archive
CHUA JINGHAO	GPT4_CHUA JINGHAO_532_Archive
DARREN TOH QING HONG	GPT4_DARREN TOH QING HONG_467_Archive
DONGJIN SHIN	GPT4_DONGJIN SHIN_719_Archive
DOUGLAS ETHAN KWOK	GPT4_DOUGLAS ETHAN KWOK_301_Archive
DUONG NGHIEP HOANG	GPT4_DUONG NGHIEP HOANG_675_Archive
EETHAN ZHANG YOU JIE	GPT4_EETHAN ZHANG YOU JIE_514_Archive
ELYSSA TJIN YI JING	GPT4_ELYSSA TJIN YI JING_142_Archive
ENZO HO	GPT4_ENZO HO_460_Archive
ERNEST GOH CHIN WEE	GPT4_ERNEST GOH CHIN WEE_869_Archive
EUGENE QUEK	GPT4_EUGENE QUEK_592_Archive
GABRIEL YU YAO SOON	GPT4_GABRIEL YU YAO SOON_978_Archive
GAN SIM RU DAYNA	GPT4_GAN SIM RU DAYNA_051_Archive
GANGADHAR NANDI GAUTHAM	GPT4_GANGADHAR NANDI GAUTHAM_850_Archive
GARG DIVYANSH	GPT4_GARG DIVYANSH_042_Archive
GARG SANCHIT AMIT	GPT4_GARG SANCHIT AMIT_679_Archive

GLAENNFJORD MILLA LINNEA ASTRID	GPT4_GLAENNFJORD MILLA LINNEA _gl_Archive
GLENNON PHUA QING XUN	GPT4_GLENNON PHUA QING XUN_609_Archive
GOH ROU HUI ASHLEY	GPT4_GOH ROU HUI ASHLEY_439_Archive
GOPINATHAN MAIYURI	GPT4_GOPINATHAN MAIYURI_060_Archive
GUPTA TANMAY	GPT4_GUPTA TANMAY_349_Archive
HAN QI-ZE	GPT4_HAN QIZE_320_Archive
HAN ZHONG DING	GPT4_HAN ZHONG DING_590_Archive
HENG YONG HEN	GPT4_HENG YONG HEN_977_Archive
HEU YU HUI	GPT4_HEU YU HUI_290_Archive
HUANG SHANGYI	GPT4_HUANG SHANGYI_564_Archive
IAN LEE JIAYI	GPT4_IAN LEE JIAYI_769_Archive
IFTIKHAR YUHAN BIN IRZAD YUHANIZAN	GPT4_IFTIKHAR YUHAN BIN IRZAD _693_Archive
ISAAC GOH YONG CHEE	GPT4_ISAAC GOH YONG CHEE_412_Archive
JAIN TANMAY	GPT4_JAIN TANMAY_216_Archive
JANG SEOK KEON	GPT4_JANG SEOK KEON_036_Archive
JAVIER LEE RUI YANG	GPT4_JAVIER LEE RUI YANG_587_Archive
JEREMIAH JAMES	GPT4_JEREMIAH JAMES_493_Archive
JORDAN RIATONO	GPT4_JORDAN RIATONO_175_Archive
JOSEFSSON CARL OSCAR WILLEHARD	GPT4_JOSEFSSON CARL OSCAR WILL_j_Archive
KANG KIAT YANG	GPT4_KANG KIAT YANG_549_Archive
KANNI KUMARAN SRISURAJ	GPT4_KANNI KUMARAN SRISURAJ_081_Archive
KARAN BONTHA	GPT4_KARAN BONTHA_031_Archive
KARTHIK SIVASUBRAMANIAN	GPT4_KARTHIK SIVASUBRAMANIAN_056_Archive
KARTIKEYAN RONITH	GPT4_KARTIKEYAN RONITH_987_Archive
KARUNANITHI NAVIN ANAND	GPT4_KARUNANITHI NAVIN ANAND_370_Archive
KAUSHAL RAJESH	GPT4_KAUSHAL RAJESH_862_Archive
KHOO KAI JUN	GPT4_KHOO KAI JUN_957_Archive
KIM DOYEON	GPT4_KIM DOYEON_415_Archive
KIM TAE-EUN	GPT4_KIM TAEEUN_919_Archive
KOH KA HOW	GPT4_KOH KA HOW_373_Archive
KOH LI WEI, OLIVER	GPT4_KOH LI WEI OLIVER_592_Archive
KOTHARI AADI	GPT4_KOTHARI AADI_808_Archive
KRONHAMN OSCAR LUDVIG	GPT4_KRONHAMN OSCAR LUDVIG_kr_Archive
KUMARESAN KUGAPRYAN	GPT4_KUMARESAN KUGAPRYAN_993_Archive
KYLE LEE YAO YU	GPT4_KYLE LEE YAO YU_663_Archive
LAM LOK SANG MATTHEW LUCAS	GPT4_LAM LOK SANG MATTHEW LUCA_294_Archive
LAM YAP TING	GPT4_LAM YAP TING_341_Archive
LAU ZHE WEN	GPT4_LAU ZHE WEN_969_Archive
LAW SHAO CHONG	GPT4_LAW SHAO CHONG_761_Archive
LEE CHANG FENG	GPT4_LEE CHANG FENG_463_Archive
LEE CHEONG WENG JUSTIN	GPT4_LEE CHEONG WENG JUSTIN_211_Archive
LEE EE XUAN	GPT4_LEE EE XUAN_562_Archive
LEE KWAN TZE	GPT4_LEE KWAN TZE_762_Archive
LEE PEI LING	GPT4_LEE PEI LING_692_Archive
LEE ZHON CHEN	GPT4_LEE ZHON CHEN_758_Archive
LEON MATTHEW WEI	GPT4_LEON MATTHEW WEI_323_Archive
LEROY KOH	GPT4_LEROY KOH_782_Archive
LEW YIK CHIN	GPT4_LEW YIK CHIN_716_Archive
LI MEIYI	GPT4_LI MEIYI_500_Archive
LI YI AN	GPT4_LI YI AN_037_Archive
LIAO YIRAN	GPT4_LIAO YIRAN_844_Archive
LIM CI ANN, ALOYSIUS	GPT4_LIM CI ANN ALOYSIUS_771_Archive
LIM JI YONG	GPT4_LIM JI YONG_851_Archive

LIM JING RUN	GPT4_LIM JING RUN_884_Archive
LIM JUN ZHAN	GPT4_LIM JUN ZHAN_890_Archive
LIM SHAU SHEE	GPT4_LIM SHAU SHEE_975_Archive
LIM SI HAN, GAVIN	GPT4_LIM SI HAN GAVIN_446_Archive
LIM ZI JING	GPT4_LIM ZI JING_395_Archive
LIU BOREN LAWRENCE	GPT4_LIU BOREN LAWRENCE_737_Archive
LOK WEI XIONG	GPT4_LOK WEI XIONG_376_Archive
LOKE JING JIE EVAN	GPT4_LOKE JING JIE EVAN_701_Archive
LOKHANDWALA ALIASGAR	GPT4_LOKHANDWALA ALIASGAR_001_Archive
LOO ZHONG EN SAMUEL	GPT4_LOO ZHONG EN SAMUEL_434_Archive
LOUISA CHUNG YU	GPT4_LOUISA CHUNG YU_654_Archive
LOW YI JIE SHERYL	GPT4_LOW YI JIE SHERYL_847_Archive
LYU JINHAN	GPT4_LYU JINHAN_276_Archive
MADHU KUMAR ANANNYA	GPT4_MADHU KUMAR ANANNYA_082_Archive
MADNANI KHUSHI	GPT4_MADNANI KHUSHI_069_Archive
MANDE NEIL ASHVINIKUMAR	GPT4_MANDE NEIL ASHVINIKUMAR_848_Archive
MANOJ AMIT	GPT4_MANOJ AMIT_044_Archive
MARCUS LEE WEI	GPT4_MARCUS LEE WEI_368_Archive
MARCUS LIM	GPT4_MARCUS LIM_380_Archive
MIRA KIM FABE	GPT4_MIRA KIM FABE_490_Archive
MOON JOONCHUL	GPT4_MOON JOONCHUL_280_Archive
MOON SI HO	GPT4_MOON SI HO_044_Archive
MUHAMMAD ASYRAF BIN ABDUL RAHIM	GPT4_MUHAMMAD ASYRAF BIN ABDUL_606_Archive
MUHAMMAD HAZIQ BIN ABDUL RAHIM	GPT4_MUHAMMAD HAZIQ BIN ABDUL _198_Archive
MUN JIA HAO	GPT4_MUN JIA HAO_276_Archive
MUTHUMANIKANTAN PAVITHRA	GPT4_MUTHUMANIKANTAN PAVITHRA_472_Archive
NANDHINI NACHIAPPAN	GPT4_NANDHINI NACHIAPPAN_089_Archive
NATARAJAN SRINIVASAN	GPT4_NATARAJAN SRINIVASAN_636_Archive
NATHAN BONG NENG QUAN	GPT4_NATHAN BONG NENG QUAN_650_Archive
NEHRU KABILLAN	GPT4_NEHRU KABILLAN_803_Archive
NEO KUANG YI JAYDEN	GPT4_NEO KUANG YI JAYDEN_533_Archive
NG YEE SUNG	GPT4_NG YEE SUNG_664_Archive
NG ZHI HAO CLEMENT	GPT4_NG ZHI HAO CLEMENT_642_Archive
NG ZHI HOU	GPT4_NG ZHI HOU_719_Archive
NG ZI LE	GPT4_NG ZI LE_452_Archive
NGUYEN DUC KHOA	GPT4_NGUYEN DUC KHOA_243_Archive
NGUYEN TAN KIET	GPT4_NGUYEN TAN KIET_449_Archive
ONG JUNYIN, JUSTIN	GPT4_ONG JUNYIN JUSTIN_114_Archive
ONG LI KIAT, ISAAC	GPT4_ONG LI KIAT ISAAC_158_Archive
ONG MING YU	GPT4_ONG MING YU_773_Archive
OOI WEN YUE	GPT4_OOI WEN YUE_036_Archive
PAI SHASHANK LALITKUMAR	GPT4_PAI SHASHANK LALITKUMAR_208_Archive
PAN QI YUAN KEVIN	GPT4_PAN QI YUAN KEVIN_471_Archive
PATEL SIDDHARTH BUNTYBHAI	GPT4_PATEL SIDDHARTH BUNTYBHAI_689_Archive
PEMMASANI PRABAKARAN RASHMETA SAI	GPT4_PEMMASANI PRABAKARAN RASH_248_Archive
PENG BOHAO	GPT4_PENG BOHAO_755_Archive
PLOGNER OSKAR NILS ALFRED	GPT4_PLOGNER OSKAR NILS ALFRED__Archive
RAJASEKARAN KRITHIKA	GPT4_RAJASEKARAN KRITHIKA_539_Archive
RAMESH GOKUL SAI	GPT4_RAMESH GOKUL SAI_662_Archive
RAVI DEEPAK	GPT4_RAVI DEEPAK_148_Archive
REYAS MOHIDEEN FAWAZ	GPT4_REYAS MOHIDEEN FAWAZ_413_Archive
REYES JUSTIN NOEL DOTIMAS	GPT4_REYES JUSTIN NOEL DOTIMAS_680_Archive
ROYSTON CHEW YI DE	GPT4_ROYSTON CHEW YI DE_430_Archive

RUBENNISHWARAAN KARUPPIAH	GPT4_RUBENNISHWARAAN KARUPPIAH_691_Archive
RYAN PRIYANK SUNJAI DHAS	GPT4_RYAN PRIYANK SUNJAI DHAS_098_Archive
RYAN VERNADOS	GPT4_RYAN VERNADOS_024_Archive
SAM LEE HUEY SAMANTHA	GPT4_SAM LEE HUEY SAMANTHA_617_Archive
SAMPATHAWADUGE NILANA IMANDIV SILVA	GPT4_SAMPATHAWADUGE NILANA IMA_372_Archive
SELVAM ANAND	GPT4_SELVAM ANAND_304_Archive
SELVASINGH OLIVER JOSEPH	GPT4_SELVASINGH OLIVER JOSEPH_070_Archive
SEOW CHENG SI	GPT4_SEOW CHENG SI_802_Archive
SHEN YUNCONG	GPT4_SHEN YUNCONG_412_Archive
SHUKLA ATHARVA	GPT4_SHUKLA ATHARVA_251_Archive
SINGARAM KARTHIKEYAN	GPT4_SINGARAM KARTHIKEYAN_091_Archive
SPENCER TING TIEW YU	GPT4_SPENCER TING TIEW YU_254_Archive
SRIRAM ANUPAMA	GPT4_SRIRAM ANUPAMA_775_Archive
STEFAN CHOO BIN HAO	GPT4_STEFAN CHOO BIN HAO_098_Archive
STEVEN MARCUS KHOO TENG EE	GPT4_STEVEN MARCUS KHOO TENG E_978_Archive
SUA ISAAC	GPT4_SUA ISAAC_138_Archive
TAN BOON LEONG	GPT4_TAN BOON LEONG_443_Archive
TAN JIN XIANG	GPT4_TAN JIN XIANG_014_Archive
TAN KAI SHENG	GPT4_TAN KAI SHENG_237_Archive
TAN SWEE KIAT	GPT4_TAN SWEE KIAT_110_Archive
TAN YONG XIANG	GPT4_TAN YONG XIANG_557_Archive
TAN YU JUN	GPT4_TAN YU JUN_046_Archive
TEE REN WEY	GPT4_TEE REN WEY_983_Archive
THAM YOU JIE	GPT4_THAM YOU JIE_411_Archive
THOW JIA LE CALEB	GPT4_THOW JIA LE CALEB_252_Archive
TIFFANY HENG	GPT4_TIFFANY HENG_973_Archive
TIU SHENG YANG	GPT4_TIU SHENG YANG_815_Archive
TOH JIAN BIN	GPT4_TOH JIAN BIN_754_Archive
TOH SIEW HEAN	GPT4_TOH SIEW HEAN_512_Archive
TRAN HOANG KHANG	GPT4_TRAN HOANG KHANG_579_Archive
TRICIA LIM ZI NING	GPT4_TRICIA LIM ZI NING_575_Archive
TRIPATHI ANOUSHKA	GPT4_TRIPATHI ANOUSHKA_062_Archive
TRISTAN LAU YUE SHEN	GPT4_TRISTAN LAU YUE SHEN_950_Archive
TRUONG PHI HUNG	GPT4_TRUONG PHI HUNG_402_Archive
VISHWAK NATTAMAI	GPT4_VISHWAK NATTAMAI_645_Archive
VISHWANATH ANNANYA	GPT4_VISHWANATH ANNANYA_037_Archive
VITO BENEDICT HALIM	GPT4_VITO BENEDICT HALIM_783_Archive
VIVEKAN KATHIRAVAN	GPT4_VIVEKAN KATHIRAVAN_236_Archive
VUPPU NAGARAJAN YUVEN RAAJ	GPT4_VUPPU NAGARAJAN YUVEN RAA_686_Archive
WAHLGREN HANNA CHRISTINA	GPT4_WAHLGREN HANNA CHRISTINA__ch_Archive
WANG QICHEN	GPT4_WANG QICHEN_536_Archive
WILL CHIN JUN YU	GPT4_WILL CHIN JUN YU_806_Archive
WONG WEI LUN	GPT4_WONG WEI LUN_308_Archive
XANDER LIM	GPT4_XANDER LIM_790_Archive
XU RUIJIE	GPT4_XU RUIJIE_009_Archive
YEW SU YI	GPT4_YEW SU YI_674_Archive
YONG CHUAN ONN	GPT4_YONG CHUAN ONN_697_Archive
YU YIFAN	GPT4_YU YIFAN_811_Archive
YUE WEI AN JOSHUA	GPT4_YUE WEI AN JOSHUA_221_Archive
ZEPH TANG ZHENKAI	GPT4_ZEPH TANG ZHENKAI_461_Archive
ZHAO ZEYU	GPT4_ZHAO ZEYU_093_Archive
ZOE TAY HUI EN	GPT4_ZOE TAY HUI EN_198_Archive
ZOU YUYANG	GPT4_ZOU YUYANG_145_Archive

FINAL CHECKLIST FOR GPT-4:

Grading is done only once. Re-grading due to not meeting the checklist below are not accepted.

- ☐ I have used the **exact and correct numbers** from my student matriculation number.
- ☐ I have named the submitted archive **according to the given template**.
- ☐ The submitted archive is **less than 5 MB** (**Delete ".sim" folder if more than 5 MB.** Penalty of up to 10% apply if ≥5 MB).
- ☐ I confirm that the uploaded **submission is the correct / latest version**, and not an older one.
- ☐ I have **downloaded my submission** before the deadline and confirmed that it was properly uploaded to canvas.
- ☐ I confirm that the **grader need not "Generate bitstream"**, as the bitstream is already there during "Program Device".
- ☐ I have carefully verified the bitstream in my **CANVAS submission**, and it works exactly as I expected.